BOOTCAMP FULL STACK · MÓDULO BASES DE DATOS · PARTE 1

# Bases de Datos con MySQL



Consulta, une y construye bases de datos relacionales: de tu primera consulta a tu propia base de datos.

Sesiones 1 a 3 · Clase online

MÓDULO BASES DE DATOS · SESIÓN 1 DE 5

# Tu primera consulta en 10 minutos



Hoy no hay teoría aburrida: vamos a hacerle preguntas a datos reales desde el primer momento.

Bootcamp Full Stack · Clase online

💬 RESPONDE EN EL CHAT: A, B, C o D

# Subes una foto a Instagram desde el móvil. Al día siguiente la ves desde el ordenador. ¿Dónde estaba guardada?

**A** En la memoria de tu móvil

**B** En el navegador del ordenador

**C** En los servidores de Instagram

**D** En el HTML de la página

CARACTERÍSTICAS

# Lo que una base de datos te garantiza

💾

## Persistencia

Los datos sobreviven a recargas, apagones y reinicios.

✅

## Integridad

Reglas que impiden datos absurdos: una edad de -5 o un pedido sin cliente.

🔒

## Seguridad

Usuarios y permisos: cada uno ve y cambia solo lo que le toca.

👥

## Concurrencia

Miles de personas leyendo y escribiendo a la vez sin pisarse.

⚡

## Consultas rápidas

Encuentra una fila entre millones en milisegundos con SQL.

🛟

## Copias y recuperación

Backups y transacciones: o se guarda todo, o no se guarda nada.

ARQUITECTURA

# Las 3 piezas de cualquier aplicación



## Frontend

Lo que ves y tocas. HTML, CSS y JS. ¡Esto ya lo sabéis hacer!



## Backend

Recibe peticiones, comprueba y decide. Lo veréis con Node.js.



## Base de datos

Guarda los datos para siempre, ordenados y seguros. ¡Esta semana!

👉 El frontend nunca habla directamente con la base de datos: siempre pasa por el backend.

ARQUITECTURA EN MOVIMIENTO

# Así viaja una petición, del botón a la base de datos



ANALOGÍA

# Una app es como un restaurante



## La sala = Frontend

Lo que ve el cliente: la carta, las mesas. Bonita, pero no cocina nada.



## El camarero = Backend

Toma nota, lleva el pedido a cocina, comprueba y trae el plato.



## La despensa = Base de datos

Donde está todo guardado, ordenado y etiquetado para encontrarlo rápido.

LO QUE YA SABÉIS

# ¿Por qué no guardo todo en un array de JavaScript?

## Array de JS / localStorage

- Al recargar la página, **desaparece**
- Solo vive en **tu navegador**
- Tus amigos **no ven** tus datos
- Buscar entre millones sería **lentísimo**

## Base de datos

- Los datos son **permanentes**
- Están en un servidor, **compartidos**
- Miles de usuarios a la vez sin pisarse
- Busca entre millones en **milisegundos**

TIPOS DE BASES DE DATOS

# Esta semana vas a conocer dos tipos



## Relacionales · MySQL

Datos en tablas, como un Excel muy ordenado.
Sesiones 1, 2 y 3.



## Documentales · MongoDB

Datos como objetos JSON (¡como en JavaScript!).
Sesiones 4 y 5.



# SQL: el idioma para hablar con las bases de datos

Se lee casi como inglés:  SELECT nombre FROM clase WHERE edad > 25

# 01

# Pregúntale a la tabla

SELECT y WHERE: elegir columnas y filtrar filas.

CONCEPTO

# Vuestras respuestas ya son una tabla

🗂️ **clase**

| nombre | edad | ciudad | serie_favorita | horas_movil |
|---|---|---|---|---|
| Laura | 23 | Madrid | Stranger Things | 5 |
| Javi | 27 | Sevilla | The Boys | 4 |
| Marta | 21 | Valencia | Los Bridgerton | 7 |
| Dani | 25 | Madrid | The Last of Us | 3 |
| Irene | 29 | Bilbao | La casa de papel | 6 |

*Datos de ejemplo: en clase usaremos los vuestros* 😉

### Tabla

Un tipo de cosa: la clase

### Fila

Una persona (un registro)

### Columna

Un dato de todas las personas

# Una tabla = un array de objetos

```js
const clase = [
  { nombre: "Laura", edad: 23 },
  { nombre: "Javi",  edad: 27 },
  { nombre: "Marta", edad: 21 }
];
```

- Cada **objeto** = una **fila**
- Cada **propiedad** = una **columna**
- Todas las filas tienen **las mismas columnas**
- La diferencia: la BD guarda millones y nunca se borran al recargar

HERRAMIENTA

# DB Fiddle: tu laboratorio de SQL



## 1. Abre el enlace

Te lo paso por el chat. Funciona en el navegador, sin instalar nada.



## 2. Panel izquierdo

La estructura y los datos. Se crea de cero cada vez que pulsas Run.



## 3. Panel derecho

Aquí escribes TUS consultas.



## 4. Pulsa Run

Los resultados salen abajo. ¿Error? Lee el mensaje: dice la línea.

# SELECT: elegir qué columnas quiero ver

```sql
-- Todas las columnas
SELECT * FROM clase;

-- Solo algunas, separadas por comas
SELECT nombre, ciudad FROM clase;
```

- `SELECT` → qué columnas
- `*` → **todas**
- `FROM` → de qué tabla
- Termina siempre con **punto y coma** `;`

# WHERE: quedarme solo con algunas filas

```sql
SELECT nombre, edad
FROM clase
WHERE edad > 25;

SELECT * FROM clase
WHERE ciudad = 'Madrid';
```

- `=` igual · `<>` distinto
- `>` `<` `>=` `<=` para números y fechas
- Textos y fechas entre **comillas simples** `'...'`
- Números **sin comillas**
- Es como el `filter` de JavaScript

💬 RESPONDE EN EL CHAT: A, B, C o D

# ¿Qué devuelve  SELECT nombre FROM clase WHERE edad > 25 ?

**A** Todas las columnas de quien tiene más de 25

**B** Solo el nombre de quien tiene más de 25

**C** El nombre de quien tiene 25 o más

**D** La edad de todos

👩‍🏫 EJEMPLO GUIADO · PASO 1/5

# ¿Quiénes somos? Toda la tabla

SQL

```sql
SELECT * FROM clase;
```

**Resultado**

| id | nombre | edad | ciudad | … |
|---|---|---|---|---|
| 1 | Laura | 23 | Madrid | … |
| 2 | Javi | 27 | Sevilla | … |
| 3 | Marta | 21 | Valencia | … |
| 4 | Dani | 25 | Madrid | … |
| 5 | Irene | 29 | Bilbao | … |
| 6 | Sergio | 22 | Málaga | … |

↳ *12 filas (se muestran 6)*

👩‍💻 EJEMPLO GUIADO · PASO 2/5

# Solo el nombre y la serie favorita

```sql
SELECT nombre, serie_favorita
FROM clase;
```

**Resultado**

| nombre | serie_favorita |
|---|---|
| Laura | Stranger Things |
| Javi | The Boys |
| Marta | Los Bridgerton |
| Dani | The Last of Us |
| Irene | La casa de papel |
| Sergio | Arcane |

↳ *12 filas (se muestran 6)*

👩‍🏫 EJEMPLO GUIADO · PASO 3/5

# ¿Quién vive en Madrid?

SQL

```sql
SELECT nombre, ciudad
FROM clase
WHERE ciudad = 'Madrid';
```

**Resultado**

| nombre | ciudad |
|---|---|
| Laura | Madrid |
| Dani | Madrid |
| Ana | Madrid |
| Iván | Madrid |

↳ *4 filas*

👩‍💻 EJEMPLO GUIADO · PASO 4/5

# ¿Quién pasa más de 5 horas al día con el móvil?

SQL

```sql
SELECT nombre, horas_movil
FROM clase
WHERE horas_movil > 5;
```

**Resultado**

| nombre | horas_movil |
|---|---|
| Marta | 7 |
| Irene | 6 |
| Sergio | 8 |
| Carla | 6 |
| Noelia | 9 |

↳ *5 filas*

👩‍💻 EJEMPLO GUIADO · PASO 5/5

# ¿A quién NO le gusta Netflix?

```sql
SELECT nombre, plataforma_favorita
FROM clase
WHERE plataforma_favorita <> 'Netflix';
```

**Resultado**

| nombre | plataforma_favorita |
|---|---|
| Javi | Prime Video |
| Dani | Max |
| Carla | Max |
| Pablo | Max |
| Raúl | Prime Video |
| Iván | Disney+ |

↳ *6 filas*

🛠️ EJERCICIO ⏱️ 15 min

# Tu turno: el catálogo de series 📺

1. Todas las series, con todas sus columnas
2. Solo el título, la plataforma y la nota
3. Las series de Netflix
4. Las series con nota mayor que 8.5
5. Las series estrenadas antes de 2015
6. Las series que NO están finalizadas



## Cómo trabajar

Abre el enlace de DB Fiddle de las series. Uno escribe y comparte pantalla, el otro dicta. Cambiad de rol a mitad.

# 02

# Filtros más listos

Combinar condiciones y buscar texto.

# AND, OR y NOT

```sql
-- Las DOS condiciones
WHERE tipo = 'Fuego' AND ataque > 80

-- AL MENOS UNA
WHERE tipo = 'Hielo' OR tipo = 'Dragón'

-- Lo contrario
WHERE NOT tipo = 'Agua'
```

- `AND` → se cumplen **las dos**
- `OR` → **al menos una**
- `NOT` → **lo contrario**
- Como `&&`, `||` y `!` en JS 😉
- Si mezclas AND y OR, usa **paréntesis**

# Atajos: IN y BETWEEN

```sql
-- En vez de muchos OR
WHERE tipo IN ('Hielo', 'Dragón', 'Hada')

-- En vez de >= y <=
WHERE velocidad BETWEEN 100 AND 120
```

- `IN (...)` → está **en la lista**
- `BETWEEN a AND b` → **entre** a y b
- BETWEEN **incluye** los extremos

# Buscar texto con LIKE

```sql
SELECT nombre FROM pokemon WHERE nombre LIKE 'Char%';
```

### 'Char%'

**Empieza por «Char»**

Charmander, Charizard

### '%chu%'

**Contiene «chu»**

Pikachu, Raichu

### '%ite'

**Termina en «ite»**

Dragonite, Dratini ❌

### '_e%'

**2ª letra es «e»**

Geodude, Gengar

`%` = cualquier cosa (o nada)  `_` = exactamente un carácter

💬 RESPONDE EN EL CHAT: A, B, C o D

# ¿Qué consulta encuentra las series cuyo título CONTIENE la palabra «noche»?

**A**

```
WHERE titulo = 'noche'
```

**B**

```
WHERE titulo LIKE 'noche'
```

**C**

```
WHERE titulo LIKE '%noche%'
```

**D**

```
WHERE titulo IN ('noche')
```

👩‍🏫 EJEMPLO GUIADO · PASO 1/6

# Pokémon de tipo Fuego con ataque mayor que 80

SQL

```sql
SELECT nombre, tipo, ataque
FROM pokemon
WHERE tipo = 'Fuego' AND ataque > 80;
```

**Resultado**

| nombre | tipo | ataque |
|---|---|---|
| Charizard | Fuego | 84 |
| Arcanine | Fuego | 110 |
| Flareon | Fuego | 130 |
| Moltres | Fuego | 100 |

↳ *4 filas*

👩‍🏫 EJEMPLO GUIADO · PASO 2/6

# Pokémon de tipo Hielo o Dragón

SQL

```sql
SELECT nombre, tipo
FROM pokemon
WHERE tipo = 'Hielo' OR tipo = 'Dragón';
```

**Resultado**

| nombre | tipo |
|---|---|
| Articuno | Hielo |
| Dratini | Dragón |
| Dragonite | Dragón |
| Garchomp | Dragón |

↳ *4 filas*

👩‍🏫 EJEMPLO GUIADO · PASO 3/6

# Lo mismo, más corto, con IN

SQL

```sql
SELECT nombre, tipo
FROM pokemon
WHERE tipo IN ('Hielo', 'Dragón');
```

💡 El resultado es idéntico al del paso anterior: IN es solo un atajo.

**Resultado**

| nombre | tipo |
|---|---|
| Articuno | Hielo |
| Dratini | Dragón |
| Dragonite | Dragón |
| Garchomp | Dragón |

↳ *4 filas*

👩‍🏫 EJEMPLO GUIADO · PASO 4/6

# Pokémon con velocidad entre 100 y 120

SQL

```sql
SELECT nombre, velocidad
FROM pokemon
WHERE velocidad BETWEEN 100 AND 120;
```

**Resultado**

| nombre | velocidad |
|---|---|
| Charizard | 100 |
| Raichu | 110 |
| Ninetales | 100 |
| Alakazam | 120 |
| Gengar | 110 |
| Zapdos | 100 |
| Mew | 100 |

↳ *9 filas (se muestran 7)*

👩‍💻 EJEMPLO GUIADO · PASO 5/6

# Pokémon cuyo nombre empieza por «Char»

SQL

```sql
SELECT nombre
FROM pokemon
WHERE nombre LIKE 'Char%';
```

**Resultado**

| nombre |
| --- |
| Charmander |
| Charmeleon |
| Charizard |

↳ *3 filas*

👩‍🏫 EJEMPLO GUIADO · PASO 6/6

# Legendarios de la 1ª generación

SQL

```sql
SELECT nombre, generacion
FROM pokemon
WHERE legendario = TRUE
  AND generacion = 1;
```

**Resultado**

| nombre | generacion |
|---|---|
| Articuno | 1 |
| Zapdos | 1 |
| Moltres | 1 |
| Mewtwo | 1 |
| Mew | 1 |

↳ *5 filas*

🛠️ EJERCICIO ⏱️ 20 min

# Tu turno: filtra las series 🎯

1. Series de Netflix con nota de 8.5 o más
2. Series de Max o de Disney+ (hazlo con OR y luego con IN)
3. Series estrenadas entre 2018 y 2020
4. Series cuyo título empieza por «The»
5. Series cuyo título contiene «casa»
6. Series españolas de Thriller o Drama
7. **Reto:** series de una sola temporada que ya estén finalizadas



## Pista

Escribe primero la consulta más simple que funcione y ve añadiendo condiciones de una en una.

# 03

# El Top 5 y la IA traductora

Ordenar, limitar... y pedirle SQL a la IA sin que te engañe.

# ORDER BY: ordenar el resultado

```sql
-- De menor a mayor (ASC)
SELECT nombre, ataque FROM pokemon
ORDER BY ataque;

-- De mayor a menor
SELECT nombre, ataque FROM pokemon
ORDER BY ataque DESC;
```

- `ASC` → ascendente (1, 2, 3 / A, B, C)
- `DESC` → descendente
- Se puede ordenar por **varias** columnas: `ORDER BY tipo, nombre`

# LIMIT y DISTINCT

```sql
-- Solo los 5 primeros
SELECT nombre, ataque FROM pokemon
ORDER BY ataque DESC
LIMIT 5;

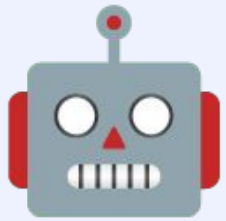
-- Sin repetidos
SELECT DISTINCT tipo FROM pokemon;
```

- `LIMIT n` → quedarse con **n filas**
- ORDER BY + LIMIT = **el Top N**
- `DISTINCT` → **quita duplicados**
- Orden: SELECT → FROM → WHERE → ORDER BY → LIMIT

COMPARATIVA

# Si sabes JavaScript, ya sabes SQL

| Quiero... | JavaScript | SQL |
|---|---|---|
| **Todo** | `pokemon` | `SELECT * FROM pokemon` |
| **Filtrar** | `pokemon.filter(p => p.ataque > 80)` | `WHERE ataque > 80` |
| **Ordenar** | `pokemon.sort((a, b) => b.ataque - a.ataque)` | `ORDER BY ataque DESC` |
| **Los 5 primeros** | `pokemon.slice(0, 5)` | `LIMIT 5` |
| **Solo el nombre** | `pokemon.map(p => p.nombre)` | `SELECT nombre FROM pokemon` |



# La IA escribe SQL muy bien… pero ¿cómo sabes si está bien?

Tu trabajo ya no es memorizar comandos: es saber pedir, leer y comprobar.

IA

# Cómo pedirle SQL a la IA



## 1. Dale contexto

Pega la estructura de la tabla (el CREATE TABLE). Sin eso, se inventa columnas.



## 2. Pregunta concreta

«Los 3 pokémon de agua más altos», no «dame pokémon».



## 3. Léela antes

¿Entiendes cada línea? ¿Usa columnas que existen?



## 4. Comprueba

¿El resultado tiene sentido? Contrástalo con algo que sepas.

🤖 PROMPT DE EJEMPLO

# Así se le pide SQL a la IA

```
Tengo esta tabla en MySQL:

CREATE TABLE pokemon (id INT, nombre VARCHAR(50), tipo VARCHAR(30), generacion INT,
altura_m DECIMAL(4,1), peso_kg DECIMAL(5,1), ataque INT, defensa INT, velocidad INT,
legendario BOOLEAN);

Escribe una consulta SQL que responda: ¿cuál es el Pokémon de tipo Agua más alto?
Explícame cada línea.
```

👉 **Contexto (la tabla) + pregunta concreta + «explícamelo»**

👩‍🏫 EJEMPLO GUIADO · PASO 1/4

# Los 5 Pokémon más rápidos

SQL

```sql
SELECT nombre, velocidad
FROM pokemon
ORDER BY velocidad DESC
LIMIT 5;
```

**Resultado**

| nombre | velocidad |
|---|---|
| Mewtwo | 130 |
| Jolteon | 130 |
| Alakazam | 120 |
| Gengar | 110 |
| Raichu | 110 |

↳ *5 filas*

👩‍🏫 EJEMPLO GUIADO · PASO 2/4

# Los 3 más pesados

SQL

```sql
SELECT nombre, peso_kg
FROM pokemon
ORDER BY peso_kg DESC
LIMIT 3;
```

**Resultado**

| nombre | peso_kg |
|---|---|
| Snorlax | 460.0 |
| Gyarados | 235.0 |
| Lapras | 220.0 |

↳ *3 filas*

👩‍🏫 EJEMPLO GUIADO · PASO 3/4

# ¿Qué tipos distintos hay?

SQL

```sql
SELECT DISTINCT tipo
FROM pokemon
ORDER BY tipo;
```

**Resultado**

| tipo |
|---|
| Agua |
| Dragón |
| Eléctrico |
| Fantasma |
| Fuego |
| Hada |
| Hielo |

↳ *14 filas (se muestran 7)*

👩‍💻 EJEMPLO GUIADO · PASO 4/4

# Con IA: «¿Cuál es el Pokémon de tipo Agua más alto?»

SQL

```sql
SELECT nombre, altura_m
FROM pokemon
WHERE tipo = 'Agua'
ORDER BY altura_m DESC
LIMIT 1;
```

**Resultado**

| nombre | altura_m |
|---|---|
| Gyarados | 6.5 |

↳ *1 fila*

💡 Comprobación: Gyarados mide 6,5 m y Lapras 2,5 m. ¡Tiene sentido!

🛠️ EJERCICIO ⏱️ 15 min 🤖 Con IA

# Tu turno: Top series + IA 🏆

1. Las 5 series con mejor nota (título y nota)
2. Las 3 series más antiguas
3. ¿Qué géneros distintos hay?
4. 🤖 Pide a la IA: «¿Cuál es la serie española con mejor nota?» (sale 1 serie)
5. 🤖 Pide a la IA: «¿Qué series de Netflix tienen más de 4 temporadas?» (salen 6)
6. 🤖 Pide a la IA: «¿Qué comedias han terminado ya?» (sale 1)



## Pasos 4 a 6

Pega en la IA el CREATE TABLE de series (panel izquierdo) y la pregunta. Ejecuta lo que te dé: ¿sale el número de filas que te digo?

MÓDULO BASES DE DATOS · SESIÓN 2 DE 5

# Muchas tablas, una respuesta



Resumir datos, unir tablas con JOIN y diseñar una base de datos con la IA... sin dejar que te la cuele.

Bootcamp Full Stack · Clase online

# 01

# Resumir datos

De 49 filas a una sola respuesta.



# «¿Cuántos clientes tenemos?» «¿Cuál es el precio medio?»

Las preguntas que más hace una empresa no piden filas: piden UN número.

AGREGACIÓN

# Funciones de resumen

SQL

```sql
SELECT COUNT(*)      FROM pokemon;   -- cuántos
SELECT SUM(peso_kg) FROM pokemon;   -- suma
SELECT AVG(ataque)   FROM pokemon;   -- media
SELECT MIN(ataque)   FROM pokemon;   -- mínimo
SELECT MAX(ataque)   FROM pokemon;   -- máximo
```

- Convierten **muchas filas en un valor**
- Como `array.length` o un `reduce` en JS
- `AS` pone **nombre** al resultado
- `ROUND(x, 1)` redondea a 1 decimal

AGREGACIÓN

# GROUP BY: hacer montones y contar cada uno

**pokemon**

| nombre | generacion |
|---|---|
| Pikachu | 1 |
| Chikorita | 2 |
| Mew | 1 |
| Lucario | 4 |
| Lugia | 2 |
| Snorlax | 1 |

`GROUP BY generacion`



**resultado**

| generacion | total |
|---|---|
| 1 | 3 |
| 2 | 2 |
| 4 | 1 |

SQL

```sql
SELECT generacion,
       COUNT(*) AS total
FROM pokemon
GROUP BY generacion;
```

AGREGACIÓN

# GROUP BY, ORDER BY y HAVING

```sql
-- Cuántos hay por generación, de más a menos
SELECT generacion, COUNT(*) AS total
FROM pokemon
GROUP BY generacion
ORDER BY total DESC;

-- Solo las generaciones con más de 5
SELECT generacion, COUNT(*) AS total
FROM pokemon
GROUP BY generacion
HAVING total > 5;
```

- La columna del `GROUP BY` va también en el `SELECT`
- `WHERE` filtra **filas** (antes de agrupar)
- `HAVING` filtra **grupos** (después)

AGREGACIÓN

# HAVING: filtrar grupos después de agrupar

### 1 · GROUP BY generacion

**generacion = 1**

Bulbasaur, Pikachu, Mew... y 37 más

`COUNT(*) = 40`

**generacion = 2**

Chikorita, Togepi, Lugia... y 4 más

`COUNT(*) = 7`

**generacion = 4**

Garchomp, Lucario

`COUNT(*) = 2`

### 2 · HAVING total > 5

40 > 5 ✓ se queda

7 > 5 ✓ se queda

2 > 5 × descartado

### 3 · Lo que devuelve

resultado

| generacion | total |
|---|---|
| 1 | 40 |
| 2 | 7 |

SQL

```
SELECT generacion,
       COUNT(*) AS total
FROM pokemon
GROUP BY generacion
HAVING total > 5;
```

💬 RESPONDE EN EL CHAT: A, B, C o D

# ¿Qué hace `SELECT tipo_id, AVG(ataque) FROM pokemon GROUP BY tipo_id`?

**A** La media de ataque de todos los Pokémon

**B** La media de ataque de cada tipo

**C** El ataque de cada Pokémon y su tipo

**D** Da error

👩‍🏫 EJEMPLO GUIADO · PASO 1/5

# ¿Cuántos Pokémon hay en total?

SQL

```sql
SELECT COUNT(*) AS total
FROM pokemon;
```

**Resultado**

| total |
| --- |
| 49 |

↳ *1 fila*

👩‍🏫 EJEMPLO GUIADO · PASO 2/5

# Ataque medio, máximo y mínimo

SQL

```sql
SELECT ROUND(AVG(ataque), 1) AS media,
       MAX(ataque) AS maximo,
       MIN(ataque) AS minimo
FROM pokemon;
```

**Resultado**

| media | maximo | minimo |
|---|---|---|
| 73.3 | 134 | 10 |

↳ *1 fila*

👩‍🏫 EJEMPLO GUIADO · PASO 3/5

# ¿Cuántos son legendarios?

SQL

```sql
SELECT COUNT(*) AS legendarios
FROM pokemon
WHERE legendario = TRUE;
```

**Resultado**

| legendarios |
| --- |
| 6 |

↳ *1 fila*

👩‍🏫 EJEMPLO GUIADO · PASO 4/5

# ¿Cuántos hay de cada generación?

SQL

```sql
SELECT generacion, COUNT(*) AS total
FROM pokemon
GROUP BY generacion;
```

**Resultado**

| generacion | total |
|---|---|
| 1 | 40 |
| 2 | 7 |
| 4 | 2 |

↳ *3 filas*

👩‍🏫 EJEMPLO GUIADO · PASO 5/5

# ¿Cuántos hay de cada tipo? (de más a menos)

SQL

```sql
SELECT tipo_id, COUNT(*) AS total
FROM pokemon
GROUP BY tipo_id
ORDER BY total DESC;
```

💡 ¿Qué es el tipo 2? ¿Y el 3? Los números no dicen nada... 🤔 (lo resolvemos en el bloque 2)

**Resultado**

| tipo_id | total |
|---|---|
| 2 | 10 |
| 3 | 9 |
| 6 | 5 |
| 1 | 4 |
| 4 | 4 |
| 5 | 3 |
| 8 | 3 |
| 12 | 3 |

↳ *14 filas (se muestran 8)*

🛠️ EJERCICIO ⏱️ 20 min

# Tu turno: estadísticas de series 📈

1. ¿Cuántas series hay en total?
2. La nota media de todas las series (2 decimales)
3. ¿Cuántas series son españolas?
4. La nota más alta y la más baja
5. Número de series por género, de más a menos
6. Nota media por país
7. Reto: géneros con más de 4 series (HAVING)



## Pista

Empieza por el COUNT sin GROUP BY y, cuando funcione, añade el GROUP BY.

# 02

# Varias tablas: claves y JOIN

Por qué los datos se separan... y cómo se vuelven a juntar.

EL PROBLEMA

# ¿Por qué no guardar el tipo como texto?

🗂️ **pokemon (mal diseño)**

| nombre | tipo |
|---|---|
| Charmander | Fuego |
| Vulpix | fuego |
| Growlithe | Fuego |
| Moltres | Fueg |
| Squirtle | Agua |

- Cada vez que escribes «Fuego» puedes **equivocarte**
- Un `GROUP BY tipo` daría **4 tipos de Fuego** distintos
- Si cambias el nombre del tipo, hay que tocar **todas las filas**
- Solución: escribirlo **una sola vez** en otra tabla

# La solución: dos tablas conectadas

🗂️ **tipos**

| id 🔑 | nombre |
|---|---|
| 1 | Planta |
| 2 | Fuego |
| 3 | Agua |



🗂️ **pokemon**

| id 🔑 | nombre | tipo_id 🔗 |
|---|---|---|
| 4 | Charmander | 2 |
| 37 | Vulpix | 2 |
| 58 | Growlithe | 2 |
| 7 | Squirtle | 3 |

### 🔑 Clave primaria (PK)

Identifica cada fila. Nunca se repite. Suele ser un id.

### 🔗 Clave foránea (FK)

Guarda la PK de OTRA tabla: tipo_id = 2 significa «Fuego».

# JOIN empareja filas como si fueran fichas

🗂️ **pokemon**

| nombre | tipo_id 🔗 |
|---|---|
| Bulbasaur | 1 |
| Charmander | 2 |
| Vulpix | 2 |
| Squirtle | 3 |



🗂️ **tipos**

| id 🔑 | nombre |
|---|---|
| 1 | Planta |
| 2 | Fuego |
| 3 | Agua |

```
pokemon.tipo_id  =  tipos.id
```

# JOIN: la sintaxis

```sql
SELECT p.nombre, t.nombre AS tipo
FROM pokemon p
JOIN tipos t ON p.tipo_id = t.id;
```

- `FROM` tabla 1 `JOIN` tabla 2
- `ON` → cómo se emparejan: **FK = PK**
- `p` y `t` son **alias** (apodos) de las tablas
- `p.nombre` = la columna nombre **de pokemon**
- Después funciona todo lo de siempre: WHERE, GROUP BY, ORDER BY...

# Encadenar varios JOIN

SQL

```sql
SELECT e.nombre AS entrenador,
       p.nombre AS pokemon,
       c.apodo
FROM capturas c
JOIN entrenadores e ON c.entrenador_id = e.id
JOIN pokemon p      ON c.pokemon_id = p.id;
```

- `capturas` conecta entrenadores con Pokémon
- Un JOIN **por cada tabla** que añades
- Cada JOIN con su propio `ON`

# JOIN vs LEFT JOIN

## JOIN

Solo las filas con pareja

🗂️ **tipos + pokemon**

| tipo | pokemon |
|---|---|
| Fuego | Charmander |
| Fuego | Vulpix |
| Agua | Squirtle |

## LEFT JOIN

TODAS las de la izquierda, tengan pareja o no

🗂️ **tipos + pokemon**

| tipo | pokemon |
|---|---|
| Fuego | Charmander |
| Fuego | Vulpix |
| Agua | Squirtle |
| Acero | NULL |

👀 **Nadie es de tipo Acero (todavía)**

💬 RESPONDE EN EL CHAT: A, B, C o D

# En la tabla capturas, ¿qué es entrenador_id?

A

La clave primaria de capturas

B

Una clave foránea que apunta a entrenadores

C

El nombre del entrenador

D

Un número aleatorio

👩‍🏫 EJEMPLO GUIADO · PASO 1/5

# Cada Pokémon con el NOMBRE de su tipo

SQL

```sql
SELECT p.nombre, t.nombre AS tipo
FROM pokemon p
JOIN tipos t ON p.tipo_id = t.id;
```

**Resultado**

| nombre | tipo |
|---|---|
| Squirtle | Agua |
| Wartortle | Agua |
| Blastoise | Agua |
| Psyduck | Agua |
| Magikarp | Agua |
| Gyarados | Agua |
| Lapras | Agua |
| Vaporeon | Agua |

↳ *49 filas (se muestran 8)*

👩‍🏫 EJEMPLO GUIADO · PASO 2/5

# Solo los de tipo Fuego

SQL

```sql
SELECT p.nombre, t.nombre AS tipo
FROM pokemon p
JOIN tipos t ON p.tipo_id = t.id
WHERE t.nombre = 'Fuego';
```

**Resultado**

| nombre | tipo |
|---|---|
| Charmander | Fuego |
| Charmeleon | Fuego |
| Charizard | Fuego |
| Vulpix | Fuego |
| Ninetales | Fuego |
| Growlithe | Fuego |
| Arcanine | Fuego |
| Flareon | Fuego |

↳ *10 filas (se muestran 8)*

👩‍🏫 EJEMPLO GUIADO · PASO 3/5

# ¿Cuántos hay de cada tipo? Ahora con nombre

SQL

```sql
SELECT t.nombre AS tipo, COUNT(*) AS total
FROM pokemon p
JOIN tipos t ON p.tipo_id = t.id
GROUP BY t.nombre
ORDER BY total DESC;
```

💡 ¡Ahora sí se entiende! Compara con el último paso del bloque 1.

**Resultado**

| tipo | total |
|---|---|
| Fuego | 10 |
| Agua | 9 |
| Psíquico | 5 |
| Eléctrico | 4 |
| Planta | 4 |
| Dragón | 3 |
| Normal | 3 |
| Roca | 3 |

↳ *14 filas (se muestran 8)*

👩‍🏫 EJEMPLO GUIADO · PASO 4/5

# ¿Qué Pokémon ha capturado Ash? (3 tablas)

SQL

```sql
SELECT e.nombre AS entrenador, p.nombre, c.nivel
FROM capturas c
JOIN entrenadores e ON c.entrenador_id = e.id
JOIN pokemon p ON c.pokemon_id = p.id
WHERE e.nombre = 'Ash';
```

**Resultado**

| entrenador | nombre | nivel |
|---|---|---|
| Ash | Pikachu | 35 |
| Ash | Bulbasaur | 22 |
| Ash | Charmander | 30 |
| Ash | Squirtle | 18 |
| Ash | Snorlax | 40 |
| Ash | Lucario | 45 |

↳ *6 filas*

👩‍🏫 EJEMPLO GUIADO · PASO 5/5

# LEFT JOIN: ¿qué tipos no tienen ningún Pokémon?

SQL

```sql
SELECT t.nombre
FROM tipos t
LEFT JOIN pokemon p ON p.tipo_id = t.id
WHERE p.id IS NULL;
```

**Resultado**

| nombre |
| --- |
| Acero |
| Bicho |

↳ *2 filas*

💡 Truco: LEFT JOIN + IS NULL = «lo que no tiene pareja».

🛠️ EJERCICIO ⏱️ 25 min

# Tu turno: une las series 🔗

1. Título de cada serie con el nombre de su plataforma
2. Las series de Netflix (filtrando por el nombre)
3. ¿Cuántas series tiene cada plataforma? De más a menos
4. Nota media por plataforma
5. Qué series ha valorado Lucía y con qué puntuación (3 tablas)
6. LEFT JOIN: plataformas que no tienen ninguna serie
7. Reto: usuarios que no han valorado ninguna serie



### Antes de empezar

Mira el panel izquierdo: ¿qué columna de series apunta a plataformas? ¿Y valoraciones a qué tablas apunta?

# 03

# Diseñar con IA

La IA dibuja el plano en segundos. Tú decides si está bien.

DISEÑO

# Diseñar = responder 3 preguntas



## Entidades

¿Qué COSAS guardo? Usuarios, canciones... → tablas



## Atributos

¿Qué DATOS tiene cada cosa? → columnas



## Relaciones

¿Cómo se CONECTAN? → claves (FK)

✏️ Truco: las entidades suelen ser los **sustantivos** del enunciado y las relaciones, los **verbos**.

RELACIONES

# ¿Cuántos con cuántos? Cardinalidades

## 1 : 1

### Uno a uno

Una persona tiene un DNI. Poco frecuente.

## 1 : N

### Uno a muchos

Un tipo tiene muchos Pokémon; cada Pokémon tiene un tipo. ¡La más habitual!

## N : M

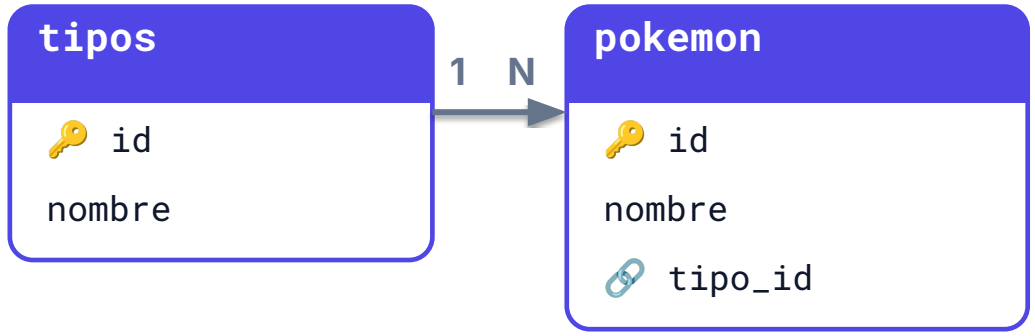
### Muchos a muchos

Un entrenador captura muchos Pokémon y un Pokémon lo capturan muchos entrenadores.

DISEÑO

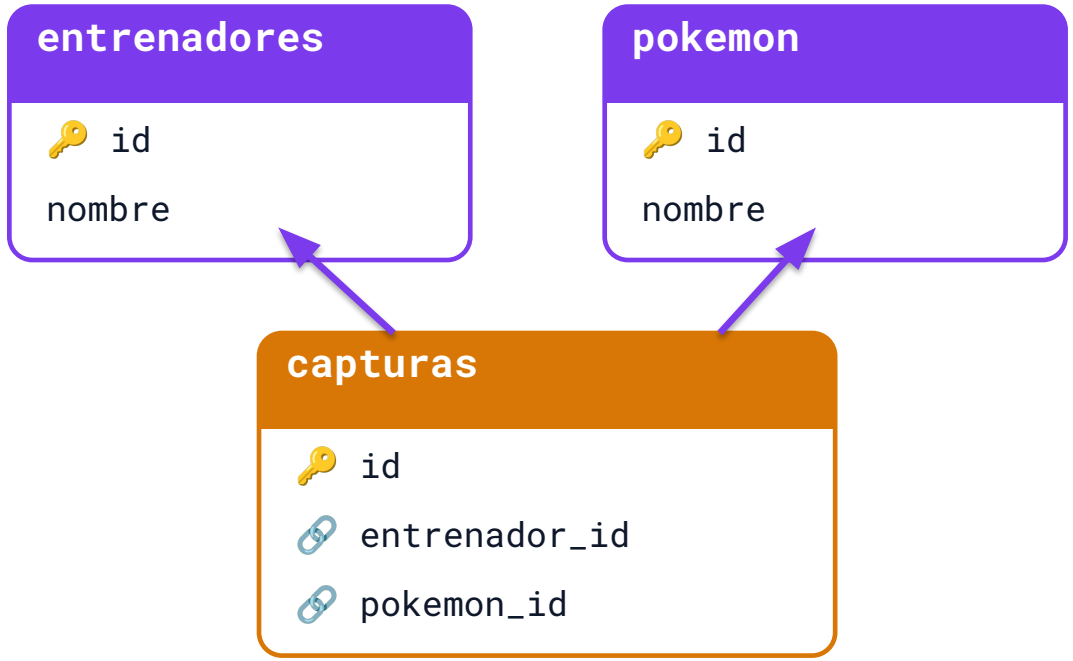
# Del dibujo a las tablas: 2 reglas de oro

## 1:N → la FK va en el lado «muchos»




Muchos Pokémon apuntan al mismo tipo. Un tipo NO puede guardar «todos sus Pokémon» en una columna.

## N:M → tabla intermedia



REVISIÓN

# Checklist para revisar el diseño de la IA ✅

🔑

### ¿Todas tienen PK?

Cada tabla con su id.

♻️

### ¿Hay datos repetidos?

Si un texto se repite mucho, quizá es otra tabla.

🔗

### ¿FK en el lado N?

La FK va en la tabla de «muchos».

🔀

### ¿N:M con intermedia?

Muchos a muchos = tabla intermedia.

📝

### ¿Cubre el enunciado?

¿Puedo responder a todo lo que pide?

🧪

### ¿Lo he probado?

Inventa 3 filas y comprueba que encajan.

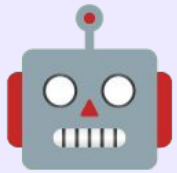
MÉTODO

# Cómo diseñar con IA



### 1. Piensa tú

Lee el enunciado y apunta entidades y relaciones



### 2. Pide el diagrama

La IA lo genera en formato Mermaid



### 3. Míralo

Pégalo en mermaid.live



### 4. Revisa y corrige

Pasa el checklist. Corrige tú o pídeselo

👩‍🏫 EJEMPLO GUIADO · PASO 1/5

# Diseñamos un «Spotify mini»

Los **usuarios** crean **playlists**. Cada playlist tiene muchas **canciones**, y una misma canción puede estar en muchas playlists. Cada canción la interpreta un **artista**, y un artista tiene muchas canciones. De cada canción queremos el título y la duración.

💬 **Por el chat: ¿qué entidades veis? ¿Qué relaciones? ¿Alguna es muchos a muchos?**

👩‍🏫 EJEMPLO GUIADO · PASO 2/5

# Le pedimos el diagrama a la IA

```
Diseña la base de datos MySQL para este enunciado: [pegar el enunciado].

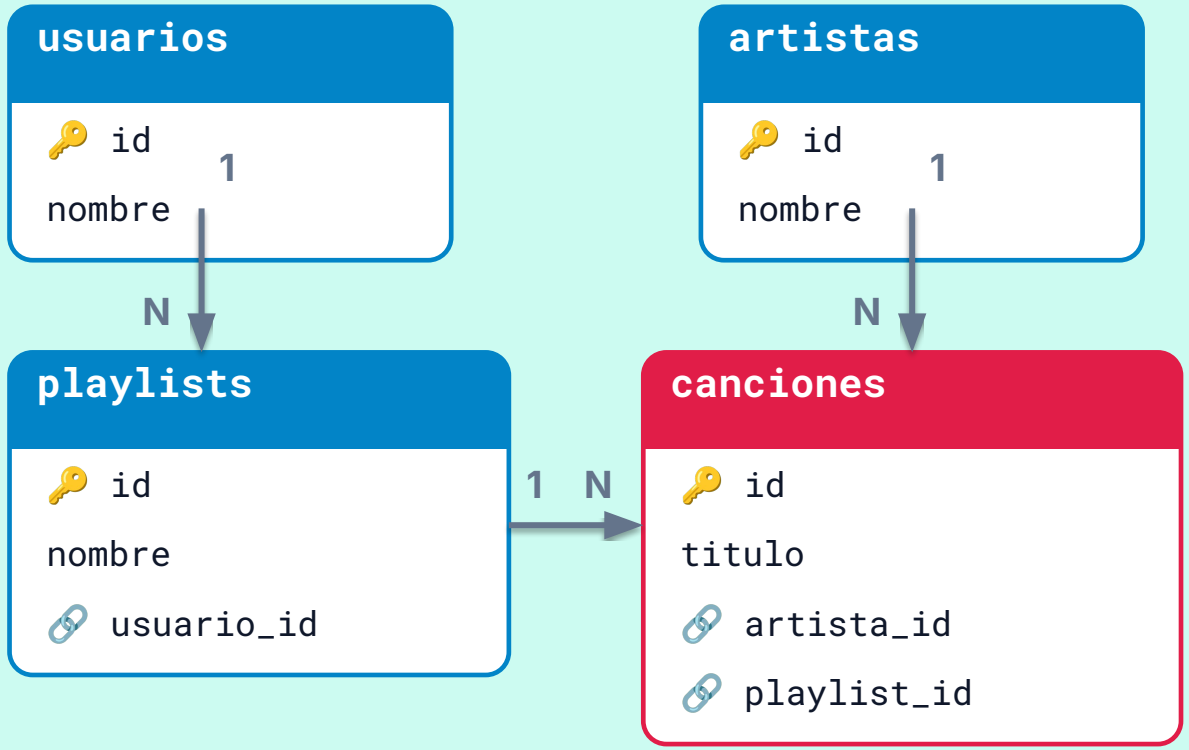
Devuélveme un diagrama erDiagram en formato Mermaid con las tablas, sus
columnas, PK y FK, y las relaciones. Usa nombres en español.
```

👉 **Copiamos su respuesta en mermaid.live para verla dibujada**

# Lo que nos devuelve la IA… ¿ves el error?

Mermaid

```
erDiagram
  USUARIO ||--o{ PLAYLIST : crea
  ARTISTA ||--o{ CANCION : interpreta
  PLAYLIST ||--o{ CANCION : contiene
  CANCION {
    int id PK
    string titulo
    int duracion_seg
    int artista_id FK
    int playlist_id FK
  }
```




🤔 **Pista: ¿puede una canción estar en DOS playlists?**

👩‍🏫 EJEMPLO GUIADO · PASO 4/5

# Pasamos el checklist

✅ Todas las tablas tienen PK

✅ No hay datos repetidos

✅ artista_id y usuario_id están en el lado N

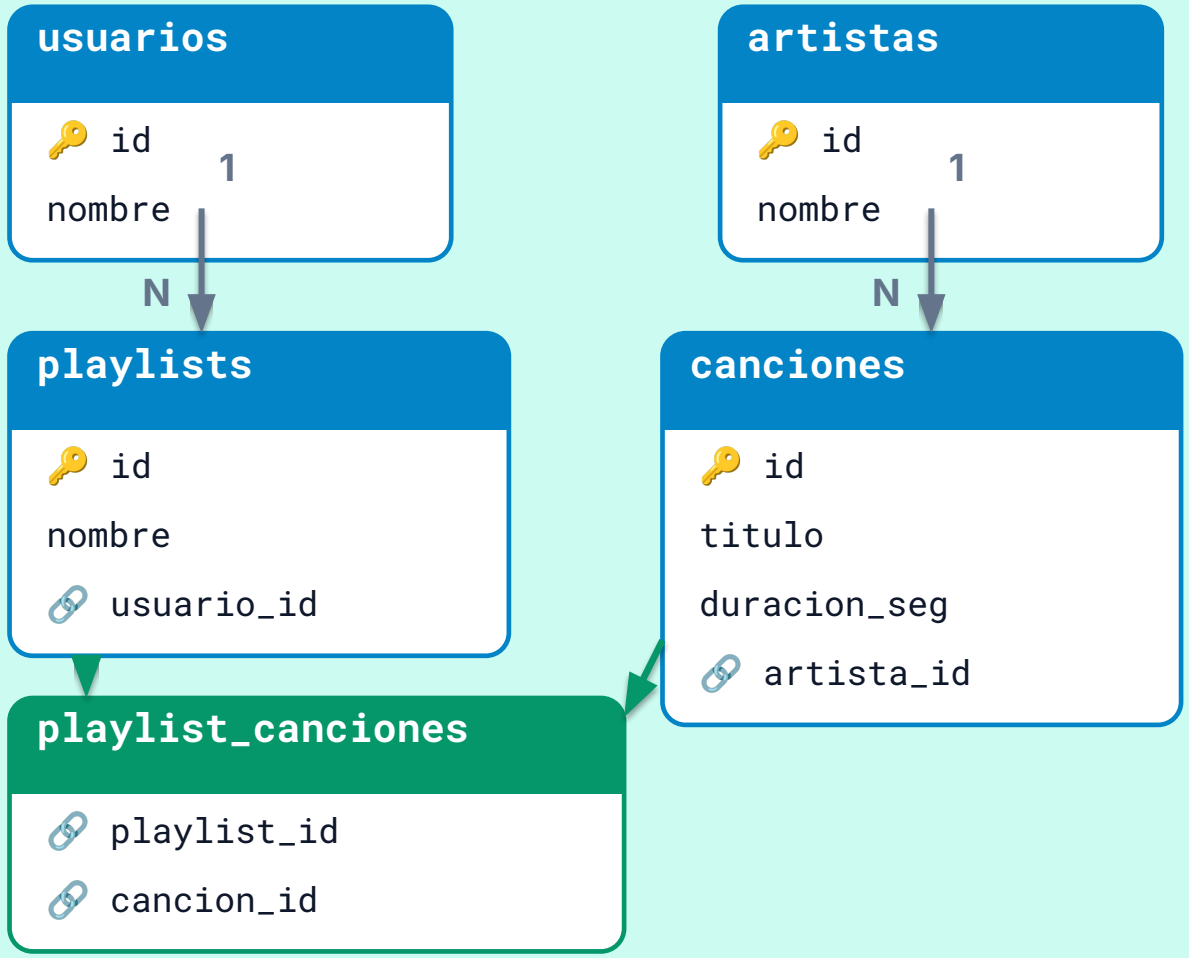
❌ Playlist ↔ Canción es **N:M** y no tiene tabla intermedia: con `playlist_id` en canciones, una canción solo puede estar en UNA playlist

✅ Cubre el resto del enunciado

# Diseño corregido

Mermaid

```
erDiagram
  USUARIO ||--o{ PLAYLIST : crea
  ARTISTA ||--o{ CANCION : interpreta
  PLAYLIST ||--o{ PLAYLIST_CANCION : contiene
  CANCION ||--o{ PLAYLIST_CANCION : aparece_en
```



🛠️ EJERCICIO ⏱️ 25 min 🤖 Con IA

# Tu turno: diseña EcoSwap con la IA ♻️

1 Lee el enunciado de **EcoSwap** (te lo paso por el chat)

2 Apunta tú primero: **entidades**, **relaciones** y **cardinalidades**

3 Pide a la IA el diagrama en **Mermaid** y míralo en **mermaid.live**

4 Pásale el **checklist**: apunta al menos un fallo o una mejora

5 Corrige el diagrama y **pega una captura** en el chat del canal



### Ojo con…

Los favoritos: un usuario marca muchos productos y un producto puede ser favorito de muchos usuarios. ¿Qué tipo de relación es?

🏠 TAREA PARA CASA

📅 4 días

# Prepárate para construir 🏗️

*Todos el mismo enunciado: GameDex, una web de videojuegos. La sesión 3 empieza corrigiéndolo.*

💾

## 1. Instala MySQL

MySQL Server + Workbench con la guía. Ejecuta pokemon.sql y SELECT COUNT(*) FROM pokemon: deben salir 49.

📄

## 2. Lee GameDex

El enunciado está en la siguiente diapositiva y en el canal. Subraya entidades y relaciones.

🤖

## 3. Diseña con IA

Usa el prompt que te doy, mira el diagrama en mermaid.live y pásale el checklist.

✅

## 4. Cinco preguntas

Respuestas cortas sobre tu diagrama. Sirven para comprobar que lo entiendes.

📦 **Entrega: Captura del COUNT = 49 + código Mermaid + tus 5 respuestas (en el canal del curso)**

TAREA · PASO 2

# GameDex: el enunciado 🎮

🎮

## Videojuegos

Título, año de lanzamiento, precio en euros, nota de la crítica (0–100) y URL de su imagen.

🏢

## Estudios

Nombre y país. Cada juego lo hace UN estudio; un estudio puede hacer MUCHOS juegos.

🏷️

## Géneros

Solo el nombre (Acción, Indie, Rol…). Un juego tiene VARIOS géneros y un género tiene MUCHOS juegos.

🏠 TAREA · PASOS 3 Y 4

# El prompt y las 5 preguntas

## 🤖 Prompt (cópialo tal cual)

```
Diseña la base de datos MySQL para este enunciado: [pega el enunciado]. Usa nombres en español, en minúsculas y sin tildes. Devuélveme un erDiagram en Mermaid con las tablas, sus columnas, la PK, las FK y las relaciones.
```

## ✅ Responde (1–2 líneas cada una)

1. ¿Cuántas tablas tiene tu diseño y cómo se llaman?
2. ¿En qué tabla va la FK del estudio y por qué ahí?
3. ¿Por qué hace falta una tabla intermedia entre juegos y géneros?
4. Hades es de Acción, Indie y Rol. ¿Cuántas filas ocupa eso en la tabla intermedia? Escríbelas.
5. ¿Qué le corregiste a la IA? Si no corregiste nada, ¿qué comprobaste del checklist?

MÓDULO BASES DE DATOS · SESIÓN 3 DE 5

# Construye tu base de datos



Del diagrama a una base de datos de verdad en tu ordenador: crear tablas, meter datos y cambiarlos sin romper nada.

Bootcamp Full Stack · Clase online

HERRAMIENTA

# Workbench en 1 minuto



## 1. Conexión

Doble clic en «Local instance». Contraseña de root.



## 2. Editor

Escribe aquí tus consultas. Guárdalas como .sql.



## 3. Ejecutar

Rayo = todo. Rayo con cursor = solo la línea actual.



## 4. Schemas

Panel izquierdo: tus bases de datos. Dale a refrescar.

HOY

# Hoy pasamos del plano a la casa



**Diseño**

Sesión 2: el diagrama



**CREATE**

Construimos las tablas



**INSERT**

Metemos los datos



**UPDATE / DELETE**

Los cambiamos sin romper nada

# 01

# CREATE TABLE

Construir la estructura: las cajas vacías.

SQL

# Crear una base de datos

```sql
SHOW DATABASES;                -- ¿qué hay?

CREATE DATABASE mipokedex;     -- crear

USE mipokedex;                 -- entrar en ella

SHOW TABLES;                   -- ¿qué tablas tiene?
```

- Una base de datos = una **carpeta** de tablas
- USE → «a partir de ahora trabajo aquí»
- En Workbench también: doble clic en el panel Schemas

SQL

# Anatomía de un CREATE TABLE

```sql
CREATE TABLE entrenadores (
  id      INT AUTO_INCREMENT PRIMARY KEY,
  nombre  VARCHAR(50) NOT NULL,
  ciudad  VARCHAR(50),
  edad    INT
);
```

- Cada línea = una **columna**
- nombre + **tipo** + **reglas**
- Columnas separadas por **comas**
- ¡Sin coma en la última!

TIPOS DE DATOS

# Los 6 tipos de datos que necesitas

## INT

**Números enteros**

Ej.: ataque, edad, id

## VARCHAR(n)

**Texto corto (máx. n)**

Ej.: nombre, email

## TEXT

**Texto largo**

Ej.: descripción

## DECIMAL(5,2)

**Decimales exactos**

Ej.: precio: 13.99

## DATE

**Fecha 'AAAA-MM-DD'**

Ej.: '2025-10-03'

## BOOLEAN

**Verdadero o falso**

Ej.: legendario: TRUE

RESTRICCIONES

# Reglas para las columnas

🔑

## PRIMARY KEY

Identifica cada fila. Única y nunca vacía.

🔢

## AUTO_INCREMENT

El id se genera solo: 1, 2, 3... ¡No lo escribas tú!

❗

## NOT NULL

Obligatorio. No puede quedar vacío.

☝️

## UNIQUE

No se puede repetir. Ideal para emails.

🎁

## DEFAULT

Valor por defecto. Ej.: DEFAULT FALSE

🔗

## FOREIGN KEY

Conecta con la PK de otra tabla.

# FOREIGN KEY: las flechas del diagrama

```sql
CREATE TABLE capturas (
  id            INT AUTO_INCREMENT PRIMARY KEY,
  nivel         INT,
  entrenador_id INT NOT NULL,
  pokemon_id    INT NOT NULL,
  FOREIGN KEY (entrenador_id) REFERENCES entrenadores(id),
  FOREIGN KEY (pokemon_id)    REFERENCES pokemon(id)
);
```

- La FK va en la tabla del lado **«muchos»**
- `REFERENCES` → **a qué tabla y columna** apunta
- La tabla apuntada tiene que **existir antes**

SQL

# ¿Me he equivocado? ALTER y DROP

```sql
-- Añadir o quitar una columna
ALTER TABLE pokemon ADD peso_kg DECIMAL(5,1);
ALTER TABLE pokemon DROP COLUMN peso_kg;

-- Borrar una tabla entera
DROP TABLE IF EXISTS capturas;

-- ⚠️ Borrar TODA la base de datos
DROP DATABASE mipokedex;
```

- `ALTER` cambia la estructura **sin perder datos**
- `DROP` elimina **para siempre**
- Truco pro: guarda tus CREATE en un **script .sql** y si algo se rompe, lo vuelves a ejecutar

💬 RESPONDE EN EL CHAT: A, B, C o D

# Quiero guardar el precio de un producto (por ejemplo 19,99 €). ¿Qué tipo uso?

A INT

B VARCHAR(10)

C DECIMAL(6,2)

D BOOLEAN

👩‍🏫 EJEMPLO GUIADO · PASO 1/4

# Creamos la base de datos y entramos en ella

SQL

```sql
CREATE DATABASE mipokedex;
USE mipokedex;
```

**Resultado**

| Database (mipokedex) |
| --- |
| mipokedex |

↳ *1 fila*

👩‍🏫 EJEMPLO GUIADO · PASO 2/4

# Creamos la tabla tipos

SQL

```sql
CREATE TABLE tipos (
  id     INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(30) NOT NULL UNIQUE
);
```

**Resultado**

| Field | Type | Null | Key | … |
|---|---|---|---|---|
| id | int | NO | PRI | … |
| nombre | varchar(30) | NO | UNI | … |

↳ *2 filas*

💡 DESCRIBE tabla; enseña su estructura: columnas, tipos y claves (PRI = primaria, MUL = foránea).

👩‍🏫 EJEMPLO GUIADO · PASO 3/4

# Creamos la tabla pokemon, conectada con tipos

SQL

```sql
CREATE TABLE pokemon (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  nombre      VARCHAR(50) NOT NULL,
  ataque      INT,
  legendario BOOLEAN DEFAULT FALSE,
  tipo_id     INT NOT NULL,
  FOREIGN KEY (tipo_id) REFERENCES tipos(id)
);
```

**Resultado**

| Field | Type | Null | Key | … |
|---|---|---|---|---|
| id | int | NO | PRI | … |
| nombre | varchar(50) | NO | | … |
| ataque | int | YES | | … |
| legendario | tinyint(1) | YES | | … |
| tipo_id | int | NO | MUL | … |

↳ *5 filas*

EJEMPLO GUIADO · PASO 4/4

# ¡Se nos olvidó la altura! La añadimos con ALTER

SQL

```sql
ALTER TABLE pokemon
ADD altura_m DECIMAL(4,1);
```

**Resultado**

| Field | Type | Null | Key | … |
|---|---|---|---|---|
| id | int | NO | PRI | … |
| nombre | varchar(50) | NO | | … |
| ataque | int | YES | | … |
| legendario | tinyint(1) | YES | | … |
| tipo_id | int | NO | MUL | … |
| altura_m | decimal(4,1) | YES | | … |

↳ *6 filas*

🛠️ EJERCICIO ⏱️ 20 min

# Tu turno: construye la base de datos de series 📺

1. Crea la base de datos miseries y entra en ella
2. Tabla plataformas: id, nombre (obligatorio y único), precio_mensual
3. Tabla series: id, titulo (obligatorio), genero, anio_estreno, nota, finalizada (por defecto FALSE) y plataforma_id (FK)
4. Comprueba las dos tablas
5. Reto: añade la columna temporadas a series



## Pista

Copia la estructura del ejemplo y cámbiala. Crea plataformas ANTES que series: series apunta a plataformas.

# 02

# INSERT

Llenar las cajas.

SQL

# INSERT: añadir filas

```sql
-- Una fila
INSERT INTO tipos (nombre)
VALUES ('Eléctrico');

-- Varias filas de golpe
INSERT INTO pokemon (nombre, ataque, tipo_id)
VALUES ('Pikachu', 55, 4),
       ('Raichu',  90, 4);
```

- El **id no se escribe**: lo pone AUTO_INCREMENT
- Textos y fechas entre **comillas simples**
- Números **sin comillas**
- Mismo **número y orden** de columnas y valores

ERRORES

# Los 4 errores más típicos al hacer INSERT 🐛

❌

## Comillas

‘Pikachu’ o "Pikachu" copiadas de Word → usa siempre 'Pikachu'

❌

## Número de columnas

3 columnas y 2 valores → Column count doesn't match

❌

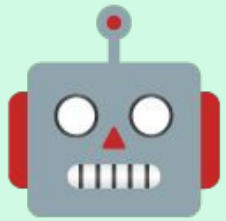
## FK que no existe

tipo_id = 99 → Cannot add or update a child row

❌

## Duplicado

Repetir un valor UNIQUE → Duplicate entry



# La IA es perfecta para generar datos de prueba...

...si le das las reglas. Si no, se inventará ids que no existen.

🤖 PROMPT DE EJEMPLO

# Generar datos con IA (bien hecho)

```
Tengo esta tabla en MySQL:
[pega aquí el CREATE TABLE pokemon]

Genera 10 INSERT con Pokémon reales de la primera generación.
Reglas: tipo_id solo puede ser 1 (Fuego), 2 (Agua) o 3 (Planta). No escribas el
id. Usa comillas simples.
```

CHECKLIST

# Antes de ejecutar lo que te da la IA, revisa:



## Las FK

¿Solo usa ids que existen?



## Las comillas

¿Simples y rectas?




## Fechas

¿Formato 'AAAA-MM-DD'?




## Columnas

¿Coinciden con tu tabla?

👩‍🏫 EJEMPLO GUIADO · PASO 1/4

# Metemos 3 tipos

SQL

```sql
INSERT INTO tipos (nombre)
VALUES ('Fuego'), ('Agua'), ('Planta');
```

**Resultado**

| id | nombre |
|---|---|
| 2 | Agua |
| 1 | Fuego |
| 3 | Planta |

↳ *3 filas*

👩‍🏫 EJEMPLO GUIADO · PASO 2/4

# Metemos un Pokémon (el id lo pone MySQL)

SQL

```sql
INSERT INTO pokemon (nombre, ataque, tipo_id, altura_m)
VALUES ('Charmander', 52, 1, 0.6);
```

**Resultado**

| id | nombre | ataque | legendario | … |
|---|---|---|---|---|
| 1 | Charmander | 52 | 0 | … |

↳ *1 fila*

👩‍🏫 EJEMPLO GUIADO · PASO 3/4

# Varios de golpe

SQL

```sql
INSERT INTO pokemon (nombre, ataque, tipo_id, altura_m)
VALUES ('Squirtle', 48, 2, 0.5),
       ('Bulbasaur', 49, 3, 0.7),
       ('Charizard', 84, 1, 1.7);
```

**Resultado**

| id | nombre | ataque | legendario | … |
|---|---|---|---|---|
| 1 | Charmander | 52 | 0 | … |
| 2 | Squirtle | 48 | 0 | … |
| 3 | Bulbasaur | 49 | 0 | … |
| 4 | Charizard | 84 | 0 | … |

↳ *4 filas*

👩‍🏫 EJEMPLO GUIADO · PASO 4/4

# ¿Y si el tipo no existe?

SQL

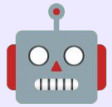
```sql
INSERT INTO pokemon (nombre, ataque, tipo_id)
VALUES ('Pikachu', 55, 99);
```

💡 MySQL dice: Cannot add or update a child row: a foreign key constraint fails (`mipokedex.pokemon`, CONSTRAINT `pokemon_i...  ¡La FK nos protege!

🛠️ EJERCICIO ⏱️ 25 min 🤖 Con IA

# Tu turno: llena tu base de datos de series 📥

1. Mete a mano 4 plataformas
2. Mete a mano 3 series
3. Pide a la IA 15 series más (respetando las FK) y ejecútalas
4. Comprueba cuántas hay
5. Comprueba que encajan: títulos con el nombre de su plataforma



## Prompt

«Dada esta tabla [CREATE TABLE series], genera 15 INSERT de series reales. plataforma_id solo puede ser 1, 2, 3 o 4. No escribas el id.»

# 03

# UPDATE y DELETE sin miedo

Con un gran poder viene una gran responsabilidad.

SQL

# UPDATE: cambiar datos

```sql
UPDATE pokemon
SET ataque = 90
WHERE id = 4;

-- Varias columnas y cálculos
UPDATE pokemon
SET ataque = ataque * 1.10, legendario = FALSE
WHERE tipo_id = 1;
```

- `SET` → qué cambia y a qué valor
- `WHERE` → **a qué filas**
- Se puede calcular con el valor anterior

# UPDATE o DELETE sin WHERE = desastre 💥

UPDATE pokemon SET ataque = 0;  →  TODOS los Pokémon con ataque 0. Y no hay botón de deshacer.

BUENAS PRÁCTICAS

# La regla de oro antes de modificar

1️⃣

## SELECT primero

Escribe un SELECT con el WHERE que vas a usar

2️⃣

## Comprueba

¿Salen solo las filas que quiero cambiar? ¿Cuántas?

3️⃣

## Cambia y ejecuta

Sustituye el SELECT * por UPDATE … SET o DELETE

Truco: usa el **id** en el WHERE siempre que puedas. Es único: imposible equivocarse de fila.

SQL

# DELETE: borrar filas

```sql
DELETE FROM pokemon
WHERE id = 3;

-- ⚠️ Borra TODAS las filas
DELETE FROM pokemon;
```

- Borra **filas**; la tabla sigue existiendo
- Si otra tabla apunta a esa fila (FK), MySQL **no te deja**
- `DELETE` ≠ `DROP`: DROP se lleva la tabla entera

WORKBENCH

# El modo seguro de Workbench 🛡️

🚫

## Error 1175

Workbench bloquea UPDATE/DELETE cuyo WHERE no use una clave (id). Es tu cinturón de seguridad.

✅

## Solución buena

Busca el id con un SELECT y usa WHERE id = ...

🔓

## Si de verdad lo necesitas

SET SQL_SAFE_UPDATES = 0;
→ tu UPDATE → SET SQL_SAFE_UPDATES = 1;

💬 RESPONDE EN EL CHAT: A, B, C o D

# ¿Qué hace  DELETE FROM series WHERE nota < 5 ?

A

Borra la tabla series

B

Borra las series con nota menor que 5

C

Pone la nota a NULL en esas series

D

Borra todas las series

👩‍🏫 EJEMPLO GUIADO · PASO 1/5

# Primero miramos: ¿qué ataque tiene Charizard?

SQL

```sql
SELECT id, nombre, ataque
FROM pokemon
WHERE nombre = 'Charizard';
```

**Resultado**

| id | nombre | ataque |
|---|---|---|
| 4 | Charizard | 84 |

↳ *1 fila*

💡 Paso 1 de la regla de oro: miramos ANTES y apuntamos su id.

👩‍🏫 EJEMPLO GUIADO · PASO 2/5

# Ahora sí: le subimos el ataque usando su id

SQL

```sql
UPDATE pokemon
SET ataque = 90
WHERE id = 4;
```

**Resultado**

| id | nombre | ataque |
|---|---|---|
| 1 | Charmander | 52 |
| 2 | Squirtle | 48 |
| 3 | Bulbasaur | 49 |
| 4 | Charizard | 90 |

↳ *4 filas*

EJEMPLO GUIADO · PASO 3/5

# Subimos un 10% el ataque de todos los de tipo Fuego

SQL

```sql
UPDATE pokemon
SET ataque = ataque * 1.10
WHERE tipo_id = 1;
```

**Resultado**

| nombre | ataque | tipo_id |
|---|---|---|
| Charmander | 57 | 1 |
| Squirtle | 48 | 2 |
| Bulbasaur | 49 | 3 |
| Charizard | 99 | 1 |

↳ *4 filas*

👩‍🏫 EJEMPLO GUIADO · PASO 4/5

# Borramos a Bulbasaur

SQL

```sql
DELETE FROM pokemon
WHERE id = 3;
```

**Resultado**

| id | nombre |
|---|---|
| 1 | Charmander |
| 2 | Squirtle |
| 4 | Charizard |

↳ *3 filas*

👩‍🏫 EJEMPLO GUIADO · PASO 5/5

# ¿Y si borro un tipo que tiene Pokémon?

```sql
DELETE FROM tipos
WHERE id = 1;
```

💡 MySQL dice: Cannot delete or update a parent row: a foreign key constraint fails (`mipokedex.pokemon`, CONSTRA… Hay Pokémon que apuntan a ese tipo.

🛠️ EJERCICIO ⏱️ 20 min

# Tu turno: modifica tus series ✏️

1. Cambia la nota de Dark a 9.0 (busca antes su id)
2. Sube 1 € el precio de Netflix
3. Marca como finalizadas las series estrenadas antes de 2015 (¡te saltará el modo seguro!)
4. Borra una serie que no te guste (por su id)
5. Intenta borrar una plataforma que tenga series. ¿Qué pasa? ¿Por qué?
6. **Bonus:** «Caza el error» (siguiente diapositiva)



## Recuerda

SELECT primero, UPDATE después. Y si te sale el error 1175… ¡lee la diapositiva del modo seguro!

BONUS

# Caza el error 🐛 (consultas «hechas por una IA»)

SQL

```sql
-- 1. Quería poner nota 9 a la serie 5
UPDATE series SET nota = 9;

-- 2. Series con nota mayor que 8
SELECT * FROM series WHERE nota > '8 puntos';

-- 3. Nueva plataforma
INSERT INTO plataformas (nombre) VALUES (Filmin);

-- 4. Borrar las series de Max (plataforma 2)
DELETE FROM series WHERE plataforma_id = '2' OR 1 = 1;
```

- ¿Qué falla en cada una?
- ¿Cuál es **la más peligrosa**?
- Moraleja: **lee** lo que te da la IA antes de ejecutarlo

🏠 TAREA PARA CASA

📅 2 días

# GameDex visual 🎮

*Construye el diseño oficial de GameDex, llénalo con datos reales y míralo convertido en una web de tarjetas.*

🏗️

## 1. Construye

Escribe TÚ los CREATE TABLE del diseño oficial en gamedex.sql (con PK y FK).

🤖

## 2. Llena

Te damos 4 CSV con datos reales de Steam. La IA los pasa a INSERT; tú compruebas los COUNT.

✨

## 3. Visualiza

Kit visor: pon tus datos y tu nombre en .env y escribe las 5 consultas fijadas en consultas.js.

🍃

## 4. Prepara Mongo

Crea tu cuenta de MongoDB Atlas (clúster M0 gratis) e instala Compass con la guía.

📦 **Entrega: Captura de tu web + gamedex.sql + captura de Compass conectado a Atlas**

🏠 TAREA · PASOS 2 Y 3

# ¿Lo has hecho bien? Compruébalo tú

## 📥 Después de cargar los CSV

`SELECT COUNT(*) FROM …`

| | |
|---|---|
| `estudios` | 21 |
| `generos` | 7 |
| `juegos` | 30 |
| `juego_genero` | 66 |

## ✨ Las 5 pestañas de tu web

| | | |
|---|---|---|
| **Catálogo** | Todos los juegos, de mejor a peor nota | 30 🃏 |
| **Menos de 15 €** | Juegos que cuestan menos de 15 €, del más barato al más caro | 11 🃏 |
| **Hecho en España** | Juegos de estudios españoles, con el nombre del estudio | 5 🃏 |
| **Indies** | Juegos del género Indie | 16 🃏 |
| **Joyas recientes** | Juegos de 2020 en adelante con nota de 90 o más, de mejor a peor nota | 6 🃏 |

⭐ **Reto: pestaña «Estudios» con cuántos juegos tiene cada estudio (21, también los que tienen 0) · Después: añade tu juego favorito**

TAREA

# Así funciona el kit visor

JS

```
// consultas.js · cada consulta = una pestaña
module.exports = {
  'Todos': `SELECT * FROM pokemon`,

  'Rápidos': `SELECT * FROM pokemon
              WHERE velocidad > 100`,

  'Top 5': `SELECT * FROM pokemon
            ORDER BY ataque DESC LIMIT 5`,
};
```

1️⃣ **npm install**

Solo la primera vez

2️⃣ **Edita .env**

Tu usuario y contraseña de MySQL

3️⃣ **npm start**

Abre http://localhost:3000

4️⃣ **Edita consultas.js**

Guarda y recarga: ¡magia! ✨